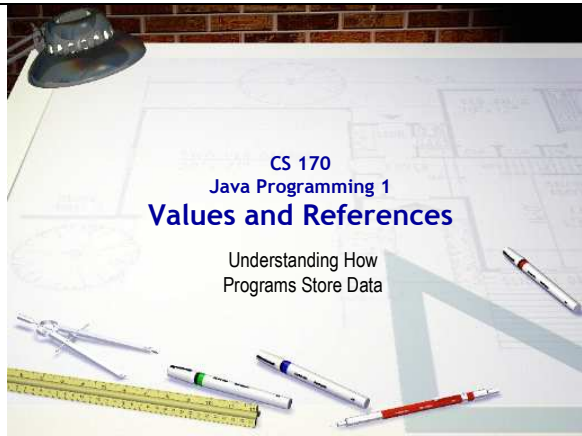




Slide 1
CS 170
Java Programming 1
Values and References

Duration: 00:00:24
Advance mode: Auto

Notes:

Hi there. This is the CS 170, Java Programming 1 lecture on Values and References. This mini-lecture was designed to accompany Section 2.10 in your textbook, titled Object References, and so it really doesn't have any hands-on exercises associated with it. In the next lesson, we'll get back to programming.



Slide 2
How Programs Store Data

Duration: 00:01:55
Advance mode: Auto

Notes:

As you learned in the first week of class, when a program runs, it uses memory to store the program's machine language instructions as well the data elements that the program manipulates, both simple numeric variables as well as objects.

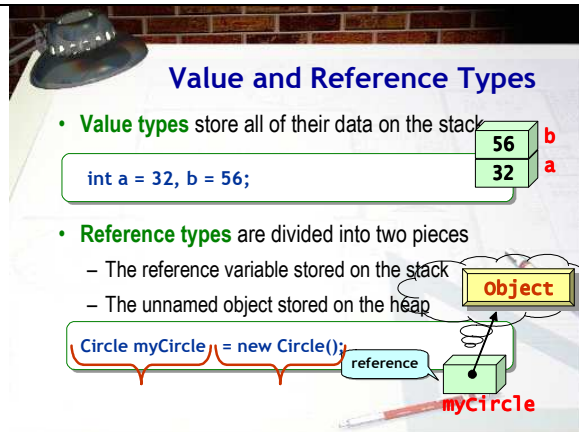
The memory reserved for data storage is further divided into three sections, designed to store three different kinds of data.

When your program is first loaded from disk into memory, there are some data items that are used as long as your program exists. These data items are loaded into an area called the static storage area. "Static" is used here in the sense of "in a fixed location". Static storage contains data that is available for the entire time a program is running and each static element stays at its initial location.

The problem with static storage is that it really limits the number of variables you can have in your program. Even if you only use a data item for a small amount of the time your program is running, the variable still takes up space in memory. The second kind of storage area, called the stack, allows your program to reuse memory space. With stack based storage, variables are assigned to a memory location when your method begins and then discarded when your method ends. That allows other methods to reuse the same physical memory for their short-lived variables. All variables that you create inside a method (like main()) use this stack-based storage.

Finally, we have the storage location where objects themselves live. This is called the heap (or free-store in C++). While local object variables live on the stack, the objects themselves, which you create using the new operator, live

outside on the heap.



Value and Reference Types

- **Value types** store all of their data on the stack

```
int a = 32, b = 56;
```

- **Reference types** are divided into two pieces
 - The reference variable stored on the stack
 - The unnamed object stored on the heap

`Circle myCircle = new Circle();`

Slide 3

Value and Reference Types

Duration: 00:01:18

Advance mode: Auto

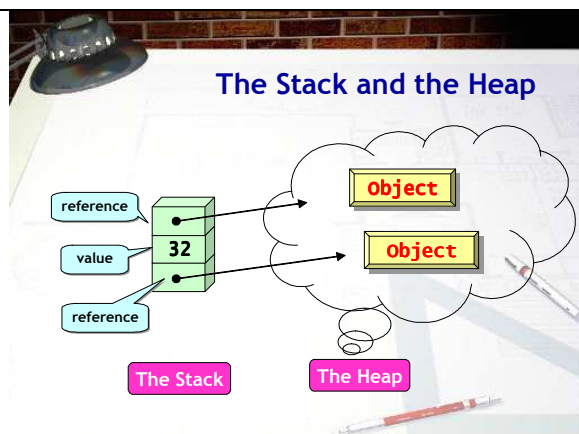
Notes:

In Chapter 3 we'll start talking about instance variables, that is, variables that are designed to hold an object's state. For right now, though, I want to concentrate on local variables: those variables that you define and use inside the methods in your programs.

When you create a local variable using one of Java's value types, such as an int, all of the variable's data is stored inside the variable itself, and that variable is stored on the stack. Here's an example, with two integer variables, a and b. In memory, those two values are stored inside a memory location, and a and b are "stacked" on top of each other like this.

Reference types, like objects and Strings, look different in memory because they have two pieces: the object variable and the object itself.

When you create an object variable with a statement like `Circle myCircle`, that variable is placed on the stack, just like the variables a and b. When you create a Circle object with `new Circle()`, an un-named Circle object is created on the heap, and a reference is then stored inside the variable myCircle which is still on the stack.



The Stack and the Heap

Slide 4

The Stack and the Heap

Duration: 00:00:25

Advance mode: Auto

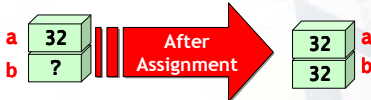
Notes:

Here's another picture that illustrates the idea in a different way. Notice that the stack can store both values, such as 32, and references. Items on the stack are stored right next to each other. Items on the heap, however, can be stored anywhere, since only the references are used to access them.

Assignment and Value Types

- Value-type vars are "buckets" holding **actual values**
 - Assignment makes duplicate, **independent** copy

```
int a = 32, b; // Two variables
b = a;         // Two independent values
```



- Often called a **deep copy**

Slide 5

Assignment and Value Types

Duration: 00:01:11

Advance mode: Auto

Notes:

This memory arrangement means that some operations, like assignment, work differently on primitive value types like numbers than they do on object types.

Since value types are like buckets or shoe-boxes that simply store values, when you assign one variable to another, the result is two independent values. Let me give you an example.

Here are two integer variables, a and b. a is given an initial value of 32 and b is left uninitialized. Then, we assign a to b. When you assign one variable to another, the assignment operator makes a copy of the value contained in the variable on the right (a in this case), and stores that copy in the variable on the left of the assignment operator (b in this case).

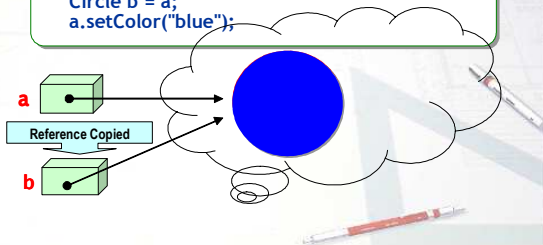
After the assignment, both a and b contain the same value (32), but they are now completely independent. If I add a third statement that now changes a to 15, it doesn't affect b at all.

This kind of assignment, where the actual value is copied is often called a deep copy.

Reference Assignment

- Copying reference types, only the **reference** is copied

```
Circle a = new Circle();
Circle b = a;
a.setColor("blue");
```



Slide 6

Reference Assignment

Duration: 00:01:26

Advance mode: Auto

Notes:

When you assign one object variable to another object variable, though, only the reference or address is copied, not the object itself. Again, a picture probably will make this a little clearer.

Assume that you create a Circle variable named a and assign it a Circle object like this. Notice that the variable a is on the stack and the red Circle object is out on the heap.

Now, let's create a second Circle variable, b, and assign a to it. Just like with value types, the contents of the variable a is copied and the copy is stored in the variable b. Because it is a reference type, though, the variable a contains a Circle reference, not a Circle object, and so only the reference is copied. After the assignment, the two variables a and b now both refer to the same red Circle object. (This is often called an "alias": two different "names" for the same actual object.)

Unlike with primitive types, these two variables are not independent. Let's see by changing a from red to blue. When you do that, notice that b also changes, because both a and b actually refer to exactly the same object.